

FACULTAD DE CIENCIAS EXACTAS, INGENIERÍA Y
AGRIMENSURA

Análisis de Lenguajes de Programación

Demostrador de Teoremas
para Lógica Modal S5 en Haskell

Autor:

Tomas Federici

31 de marzo de 2026

Índice

1. Breve introducción al sistema S5	2
1.1. Sintaxis de las fórmulas	2
1.2. Semántica de las fórmulas	2
1.3. Ejemplos	3
1.3.1. $\Box A \rightarrow A$	3
1.3.2. $\Diamond A \rightarrow A$	3
2. Sintaxis del lenguaje	4
2.1. Sintaxis concreta	4
2.2. Sintaxis abstracta	5
3. Lenguaje de dominio específico	5
3.1. Tipos de dato y mónada utilizados	5
3.2. Proceso de demostración	7
3.2.1. Casos directos	8
3.2.2. Casos modales	8
3.2.3. Casos que ramifican la demostración	9
3.3. Loop infinito y condición de bloqueo	9
4. Guía de Uso del Intérprete	10
4.1. Menú Principal	10
4.2. Modo Interactivo	10
4.3. Procesamiento de Archivos	11
4.4. Interpretación de los Resultados	11

1. Breve introducción al sistema S5

1.1. Sintaxis de las fórmulas

En el sistema S5, los operadores disponibles para construir las fórmulas constan de los operadores de la lógica de predicados más 2 operadores modales, $\Diamond$ (Operador de Posibilidad) y $\Box$ (Operador de Necesidad).

Luego, dadas A y B dos fórmulas, las siguientes fórmulas pertenecen al sistema S5:

- $\top$
- $\perp$
- $\mathbb{P}_n$, son las fórmulas atómicas
- $\neg A$
- $A \wedge B$
- $A \vee B$
- $A \rightarrow B$
- $\Diamond A$ (Es posible A)
- $\Box A$ (Es necesario A)

1.2. Semántica de las fórmulas

Para poder dar sentido a las fórmulas, en este sistema se definen los modelos como $\mathcal{M} = (W, P)$ donde W es un conjunto potencialmente infinito de mundos y P es una abreviación para una secuencia infinita de subconjuntos de W , $P = (P_1, P_2, \dots)$

Dado un modelo $\mathcal{M} = (W, P)$ y un mundo $\alpha \in W$, decimos que una fórmula A es válida en el modelo $\mathcal{M}$ y en el mundo α y lo notamos $\models_{\alpha}^{\mathcal{M}} A$, y si una fórmula es válida para todo mundo de todo modelo, simplemente decimos que la fórmula es válida y lo notamos $\models A$ (para decir que una fórmula NO es válida en un modelo y un mundo, escribimos 'no $\models_{\alpha}^{\mathcal{M}} A$ ').

Sean $\mathcal{M} = (W, P)$, $\alpha \in W$, A y B dos fórmulas, entonces:

1. $\models_{\alpha}^{\mathcal{M}} \top$
2. no $\models_{\alpha}^{\mathcal{M}} \perp$
3. $\models_{\alpha}^{\mathcal{M}} P_n \iff \alpha \in P_n$
4. $\models_{\alpha}^{\mathcal{M}} \neg A \iff \text{no } \models_{\alpha}^{\mathcal{M}} A$
5. $\models_{\alpha}^{\mathcal{M}} A \wedge B \iff \models_{\alpha}^{\mathcal{M}} A \text{ y } \models_{\alpha}^{\mathcal{M}} B$

6. $\models_{\alpha}^{\mathcal{M}} A \vee B \iff \models_{\alpha}^{\mathcal{M}} A \text{ o } \models_{\alpha}^{\mathcal{M}} B \text{ o ambos}$
7. $\models_{\alpha}^{\mathcal{M}} A \rightarrow B \iff \text{si } \models_{\alpha}^{\mathcal{M}} A \text{ entonces } \models_{\alpha}^{\mathcal{M}} B$
8. $\models_{\alpha}^{\mathcal{M}} \Diamond A \iff \exists \beta \in W / \models_{\beta}^{\mathcal{M}} A$
9. $\models_{\alpha}^{\mathcal{M}} \Box A \iff \forall \beta \in W / \models_{\beta}^{\mathcal{M}} A$

Nota sobre la semántica de Kripke y la relación de accesibilidad:

En la semántica de Kripke estándar para lógicas modales, un modelo se define habitualmente como una terna $\mathcal{M} = (W, R, P)$, donde $R \subseteq W \times W$ es la *relación de accesibilidad* entre los mundos. Sin embargo, la característica definitoria del sistema S5 es que su relación de accesibilidad es una relación de equivalencia (reflexiva, simétrica y transitiva). En términos prácticos, esto significa que todos los mundos son mutuamente accesibles entre sí (una relación universal). Por este motivo, la relación R se vuelve implícita y podemos definir el modelo únicamente mediante sus mundos y la función de valuación, es decir, $\mathcal{M} = (W, P)$.

1.3. Ejemplos

Veamos un ejemplo donde una fórmula es válida y otro donde obtenemos un contraejemplo.

1.3.1. $\Box A \rightarrow A$

Veamos que esta fórmula es válida, o sea, $\models \Box A \rightarrow A$, o lo que es equivalente, $\forall \mathcal{M} = (W, P)$ y $\forall \alpha \in W$, $\models_{\alpha}^{\mathcal{M}} \Box A \rightarrow A$.

Sean $\mathcal{M} = (W, P)$ y $\alpha \in W$

$$\models_{\alpha}^{\mathcal{M}} \Box A \rightarrow A \iff_{(7)} \text{si } \models_{\alpha}^{\mathcal{M}} \Box A \text{ entonces } \models_{\alpha}^{\mathcal{M}} A$$

Entonces, supongamos que $\models_{\alpha}^{\mathcal{M}} \Box A$; luego,
 $\models_{\alpha}^{\mathcal{M}} \Box A \iff_{(9)} \forall \beta \in W / \models_{\beta}^{\mathcal{M}} A$

En particular, para $\beta = \alpha$, $\models_{\alpha}^{\mathcal{M}} A$
 Por lo tanto, $\models_{\alpha}^{\mathcal{M}} A$

Luego, $\forall \mathcal{M} = (W, P)$ y $\forall \alpha \in W$ $\models_{\alpha}^{\mathcal{M}} \Box A \rightarrow A$
 Por lo tanto, $\models \Box A \rightarrow A$

1.3.2. $\Diamond A \rightarrow A$

Veamos que esta fórmula no es válida; para ello, debemos encontrar un modelo $\mathcal{M} = (W, P)$ y un mundo $\alpha \in W$ donde no sea válido $\models_{\alpha}^{\mathcal{M}} \Diamond A \rightarrow A$

Consideremos el siguiente modelo, $\mathcal{M} = (W, P)$ con $A = \mathbb{P}_0$ y $P_0 = \{\beta\}$

Así, $\models_{\alpha}^{\mathcal{M}} \Diamond A$, pero no $\models_{\alpha}^{\mathcal{M}} A$
 Por lo tanto, no vale $\models_{\alpha}^{\mathcal{M}} \Diamond A \rightarrow A$

2. Sintaxis del lenguaje

- El orden de precedencia de los operadores de menor a mayor y cómo asocian es:

- $\rightarrow$ (asociativo a derecha)
- $\vee$ (asociativo a izquierda)
- $\wedge$, (asociativo a izquierda)
- $\neg, \Diamond, \Box$

2.1. Sintaxis concreta

```

Form ::= var (String)
      | 'T'
      | 'F'
      | '~' Form
      | '[' Form
      | '< >' Form
      | Form '&&' Form
      | Form '||' Form
      | Form '->' Form
      | '(' Form ')'

```

Ejemplos de fórmulas bien formadas

A continuación, se presentan ejemplos de cómo traducir fórmulas lógicas a la sintaxis que reconoce el *parser* del programa:

- Axioma de Necesidad (T):** $\Box p \rightarrow p$
 Se traduce a: `[]p ->p`
- Axioma Euclidiano (5):** $\Diamond p \rightarrow \Box \Diamond p$
 Se traduce a: `<>p ->[] <>p`
- Combinación de operadores y paréntesis:** $\neg \Box p \vee \Diamond (q \wedge r)$
 Se traduce a: `~[]p || <>(q && r)`
- Distribución de la necesidad (Axioma K):** $\Box (p \rightarrow q) \rightarrow (\Box p \rightarrow \Box q)$
 Se traduce a: `[] (p ->q) ->([]p ->[]q)`

2.2. Sintaxis abstracta

$$\begin{array}{ll} \text{FormI} & ::= \text{FormI} \rightarrow \text{FormO} \\ & | \text{FormO} \\ \text{FormO} & ::= \text{FormY} \vee \text{FormO} \\ & | \text{FormY} \\ \text{FormY} & ::= \text{Base} \wedge \text{FormY} \\ & | \text{Base} \\ \text{Base} & ::= \neg \text{Base} \\ & | \Diamond \text{Base} \\ & | \Box \text{Base} \\ & | \text{Atom} \\ \text{Atom} & ::= \top \\ & | \perp \\ & | \text{Var} \\ & | (\text{FormI}) \end{array}$$

Definida así se logra:

1. **Eliminar la ambigüedad:** Se codifica el orden de precedencia directamente en las reglas de derivación. Como el analizador intenta derivar primero **FormI** y desciende hasta **Base**, los operadores del nivel más bajo (como $\neg$, $\Box$, $\Diamond$) se agrupan primero, garantizando la precedencia correcta.
2. **Asociatividad:** Permite definir de manera natural la asociatividad a izquierda de operadores como el $\wedge$ y el $\vee$, y a derecha para la implicación, facilitando su traducción directa a combinadores de *parsing* funcionales (como la librería **Parsec** en Haskell).

3. Lenguaje de dominio específico

Nota: Una breve guía de cómo compilar y correr el programa se encuentra en `README.md` del directorio del proyecto.

3.1. Tipos de dato y mónada utilizados

Por un lado, el tipo de dato **Form**, define los constructores de los operadores lógicos descritos anteriormente.

```
data Form = FTrue | FBottom
          | Var   String
          | Not   Form
          | And   Form Form
          | Or    Form Form
          | Implies Form Form
          | Dia   Form
          | Box   Form deriving (Show, Eq)
```

Fragmento de código de `src/Types.hs`

También, se tiene el tipo de dato relacionado al estado del programa o de la demostración.

```
type World = Int
type LabeledForm = (World, Form)

data StateChart = SC [LabeledForm]
                    [LabeledForm]
                    [Form]
                    [World]
                    World
                    deriving Show
```

Fragmento de código de src/Types.hs

Primero, se eligió representar un mundo con el tipo `world` como un entero y a la expresión $\models_{\alpha}^M A$ como una tupla (n, A) con el tipo `LabeledForm`.

Luego, nuestro tablero de demostración tiene:

- Una lista de fórmulas pendientes por procesar.
- Una lista de fórmulas literales, o sea, fórmulas del tipo P o $\neg P$; a través de estas podremos hallar contradicciones, por ejemplo, si llegamos a que P y $\neg P$ deben ser válidas en el mismo mundo, lo cual no es posible.
- Una lista de fórmulas universales, o sea, que valen para todo mundo; su propósito es facilitar el proceso de encontrar las fórmulas que deben heredarse a un mundo al momento de crearse.
- Una lista de los mundos creados hasta el momento. Su propósito es facilitar el proceso de agregar fórmulas universales a mundos recién creados.
- Un contador del último mundo creado, con el fin de saber cuál es el siguiente mundo a crear.

Nota: cuando hablamos de crear un mundo, no es en el sentido literal, sino que es la necesidad de empezar a considerarlo cuando una demostración necesita más de un mundo.

Por otro lado, la mónada la definimos en base al siguiente tipo y definimos las funciones `return` y `(>>=)` de la siguiente manera:

```
newtype State s m a = State {runState :: s -> m (a, s)}

instance Monad m => Monad (State s m) where
    return x = State (\s -> return (x,s))
    m >>= f = State (\s -> let t = runState m s
                           in do (v,s') <- t
                           runState (f v) s')
```

Fragmento de código de src/Monad.hs

La mónada `State` se implementa con un transformador de mónadas.

Además de las clases básicas como `Functor` y `Applicative`, se definió la clase `MonadTrans`

o mónada de transición, la cual cuenta con la función `lift` que 'eleva' una mónada dentro de otra.

```
class MonadTrans t where
  lift :: Monad m => m a -> t m a

instance MonadTrans (State s) where
  lift m = State (\s -> do a <- m
                        return (a,s))
```

Fragmento de código de `src/Monad.hs`

Se tienen, también, las funciones `getState` y `putState` para obtener y actualizar el estado respectivamente.

```
getState :: Monad m => State s m s
getState = State (\s -> return (s,s))

putState :: Monad m => s -> State s m ()
putState s = State (\_ -> return ((),s))
```

Fragmento de código de `src/Monad.hs`

Por último, se definió nuestro probador de teoremas instanciando el estado de la mónada `State` con `StateChart` y la mónada dentro de `State` como listas (`[]`).

```
type Prover a = State StateChart [] a
```

Fragmento de código de `src/Monad.hs`

3.2. Proceso de demostración

El proceso de demostración implementado es el siguiente:

Dada una fórmula F del sistema S5, se supone válida $\neg F$ en el mundo 0 y se reduce la expresión aplicando reglas lógicas y haciendo uso del `StateChart` para intentar encontrar algún mundo donde deban ser válidas dos fórmulas de la forma P y $\neg P$. Esto significaría que llegamos a una contradicción por suponer válida la fórmula $\neg F$ y, por lo tanto, debe ser válida F .

(Esto último se puede afirmar, pues el sistema S5 es un sistema de lógica consistente, y debido a esto, podemos afirmar que si una fórmula es una contradicción, luego su negación debe ser una tautología; más aún, también es completo, es decir que toda fórmula semánticamente válida es demostrable).

Sabiendo esto, veamos el procedimiento del Demostrador (evaluador implementado en `src/Eval.hs`).

Podemos separar este proceso en 3 casos:

- Los casos directos.
- Los casos modales.
- Los casos que ramifican la demostración.

Nota: Para analizar los siguientes casos, recordemos la estructura **StateChart** vista en 3.1.

3.2.1. Casos directos

Estos casos son los más simples; actualizamos el estado y seguimos la demostración. A estos casos los llamo 'directos' pues solo requieren actualizar las listas de pendientes o literales, y no se involucran en la creación de diferentes mundos.

Sean P una variable proposicional, A y B dos fórmulas; luego:

- $P \mid \neg P$: Al encontrar una variable proposicional o la negación de una de estas como válidas en un mundo n , se agregan a la lista de fórmulas literales y se prosigue con el procesamiento de las fórmulas pendientes.
- $A \wedge B$: Si esta expresión es válida en un mundo n , significa que ambas A y B deben ser válidas en el mundo n ; por lo tanto, se agregan ambas a la lista de pendientes y se continúa el proceso de las fórmulas pendientes.
- $\neg(A \vee B)$: Esta fórmula es lógicamente equivalente a $\neg A \wedge \neg B$; por lo tanto, si esta expresión es válida en un mundo n , se agregan $\neg A$ y $\neg B$ como válidas en el mundo n a la lista de pendientes y se siguen procesando los pendientes.
- $\neg(A \rightarrow B)$: Esta expresión es equivalente a $A \wedge \neg B$, por lo que, nuevamente, si esta es válida en un mundo n , se agregan A y $\neg B$ a las fórmulas pendientes en el mundo n .

3.2.2. Casos modales

Estos casos se refieren a las fórmulas que tienen operadores modales ($\Diamond$, $\Box$). Como sabemos, una fórmula $\Diamond A$ plantea la validez de la fórmula A en algún mundo que no necesariamente es el mundo donde vale $\Diamond A$ y, por otro lado, una fórmula $\Box A$ plantea la validez de la fórmula A en todos los mundos.

Sean A y B dos fórmulas; luego, los casos son:

- $\Diamond A$: La validez de este caso en un mundo n indica que la fórmula A es válida para algún mundo m . Por lo tanto, lo que se hace es sumar 1 al mundo actual (digámosle k), agregar k a la lista de mundos y poner en fórmulas pendientes a A como válida en el mundo k , más todas las fórmulas universales como válidas en el mundo k .
- $\neg\Box A$: Esta expresión es equivalente a $\Diamond\neg A$, por lo que el procedimiento es análogo al descrito anteriormente, con la diferencia de que agregamos $\neg A$.
- $\Box A$: Esta fórmula indica que A es válida en todo mundo; por lo tanto, agregamos a la lista de pendientes la fórmula A como válida para cada mundo que hayamos creado.
- $\neg\Diamond A$: En este caso, la fórmula es equivalente a $\Box\neg A$, por lo que el procedimiento es análogo al descrito anteriormente, con la diferencia de que agregamos $\neg A$.

3.2.3. Casos que ramifican la demostración

Estos casos plantean la posibilidad de que, de dos fórmulas, solo sea válida una de ellas (generalmente son los casos que tienen un operador $\vee$ o equivalentes).

La idea de implementación de esta ramificación se lleva a cabo usando la función ($>>=$) de la mónada lista, la cual es equivalente a un `concatMap`, y la función `lift` que 'sube' la lista a la mónada `State`.

- $A \vee B$: Si esta fórmula es válida en un mundo n , lo que se hace es crear dos copias del tablero actual; en una se pone como fórmula pendiente A en el mundo n y en el otro, B en el mundo n , y se ejecuta el siguiente fragmento de código:

```
do
  ...
  b <- lift [branch1, branch2]
  putState b
  checkPends
```

Fragmento de código de `src/Eval.hs`

Lo que hace esto es actualizar el estado actual con el tablero `branch1` y ejecutar `checkPends` (esta función es la que continúa expandiendo la demostración). Una vez termine de procesar toda esa rama, el resultado será `[]` si se llegó a una contradicción o el contraejemplo en caso contrario. Luego se repite el procedimiento con `branch2` y los resultados se concatenan.

- $\neg(A \wedge B)$: Esta fórmula es equivalente a $\neg A \vee \neg B$, por lo que se repite el procedimiento descrito anteriormente.
- $A \rightarrow B$: Esta expresión es equivalente a $\neg A \vee B$, y nuevamente, se repite el proceso de ramificación.

3.3. Loop infinito y condición de bloqueo

Consideremos la siguiente fórmula: $\Box \Diamond A$. Si trazamos los pasos que realiza el evaluador, nos encontramos con el siguiente escenario:

- Primero, el sistema procesa $(0, \Box \Diamond A)$, lo que indica que la fórmula es válida en el mundo inicial 0. Al evaluar el operador $\Box$, la subfórmula $\Diamond A$ se añade a la lista de fórmulas universales. La lista de tareas pendientes queda entonces con $[(0, \Diamond A)]$.
- Luego, al procesar $(0, \Diamond A)$, la semántica del operador $\Diamond$ nos obliga a crear un mundo nuevo (el mundo 1) donde A sea verdadera. Además, por estar evaluando en lógica S5, las fórmulas universales también deben aplicarse a este nuevo mundo. La lista de pendientes se actualiza a $[(1, \Diamond A), (1, A)]$.
- Después, el evaluador toma $(1, \Diamond A)$ de la lista. Al igual que antes, el operador $\Diamond$ fuerza la creación de otro mundo nuevo (el mundo 2), arrastrando nuevamente las reglas universales. La lista de pendientes pasa a ser $[(2, \Diamond A), (2, A), (1, A)]$.
- Al intentar resolver $(2, \Diamond A)$, el ciclo se repite indefinidamente.

Como podemos observar, esta dinámica genera un bucle sin fin, evaluando $\Diamond A$ y creando mundos nuevos hacia el infinito.

El origen de este problema radica en que las fórmulas con el operador $\Diamond$ son las únicas capaces de exigir la creación de nuevos mundos. En principio, la instrucción de $\Diamond A$ es simple: garantizar la existencia de algún mundo donde valga A .

Por lo tanto, la solución consiste en revisar si ya existe un mundo generado donde la fórmula A sea válida. En tal caso, no es necesario crear uno nuevo, ya que la condición de satisfactibilidad impuesta por $\Diamond A$ ya se encuentra cumplida.

```
evalForm (_, Dia f) = do
  (SC pend lits univ wlds next) <- getState
  if (checkForLocks f lits) || (checkForLocks f pend)
  then checkPends
  else let newWorld = next + 1
        heritage = map (\form -> (newWorld,form)) univ
        newPends = (newWorld,f) : heritage ++ pend
        in do putState (SC newPends lits univ (newWorld:wlds) newWorld)
              checkPends
```

Fragmento de código: Condición de bloqueo para el operador de Posibilidad

La función `checkForLocks`, simplemente busca en las listas de pendientes y literales, para ver si la fórmula A ya es válida en algún mundo, y, en ese caso, simplemente seguir con la demostración; si no la encuentra, realiza todo el proceso visto anteriormente para $\Diamond A$.

4. Guía de Uso del Intérprete

El demostrador automático se ejecuta desde la línea de comandos y presenta una interfaz de texto diseñada para ser intuitiva. Al iniciar la ejecución, el usuario se encuentra con el menú principal, el cual ofrece dos modos de funcionamiento.

4.1. Menú Principal

El menú inicial permite seleccionar la forma en la que se ingresarán las fórmulas al sistema:

- **Opción 1 - Interfaz interactiva:** Permite ingresar fórmulas manualmente una por una y evaluarlas en tiempo real.
- **Opción 2 - Cargar un archivo:** Permite evaluar un lote de fórmulas predefinidas almacenadas en un archivo de texto.
- **Comando exit:** Finaliza la ejecución del programa de manera segura.

4.2. Modo Interactivo

Al seleccionar la opción 1, el intérprete entra en un bucle interactivo de lectura y evaluación. El usuario visualizará el indicador `>` esperando el ingreso de una cadena de caracteres.

En este modo, se aplican las siguientes reglas:

- Se debe ingresar una fórmula respetando la sintaxis definida en la sección anterior (por ejemplo: $\neg p \rightarrow p$).
- **Comando help:** Imprime en pantalla una guía rápida de los símbolos aceptados por el analizador sintáctico y el orden de precedencia, sin salir del modo interactivo.
- **Comando exit:** Retorna al usuario al menú principal.

4.3. Procesamiento de Archivos

Al seleccionar la opción 2, el sistema solicitará el nombre de un archivo. El intérprete buscará automáticamente este archivo dentro del subdirectorio local `examples/`.

Para que el archivo sea procesado correctamente por el analizador, debe cumplir con los siguientes requisitos:

- Debe ser un archivo de texto plano (por ejemplo, `form.txt`).
- Cada línea del archivo debe contener exactamente una fórmula bien formada.
- Las líneas vacías o que contengan únicamente espacios en blanco serán ignoradas de forma automática, lo que permite al usuario organizar visualmente las baterías de prueba.

4.4. Interpretación de los Resultados

Para cada fórmula procesada, ya sea tipeada interactivamente o leída desde un archivo, el motor imprimirá uno de los siguientes resultados:

1. **Teorema válido:** Si la fórmula es lógicamente válida en todos los modelos de la clase S5, el sistema indicará que es un teorema de manera afirmativa.
2. **Contraejemplo:** Si la fórmula evaluada no es un teorema válido, el sistema indicará el fallo y procederá a imprimir el modelo encontrado por el algoritmo de las ramas. Este resultado mostrará el conjunto de mundos generados y la valuación específica de las variables proposicionales en cada mundo que hacen que la fórmula original sea falsa.
3. **Error de Sintaxis:** Si la cadena de caracteres ingresada no respeta la gramática formal del lenguaje, se muestra el error generado por `Parsec`